Table of Contents

Preface **1**
Three reasons 1
So why should I read this book? 3
RISC-V and bare metal? 4
What is in the book? 5
What about the hardware costs? 7
Where is the code and how to use this book? 9
What about AI? 10
What about errors? 11
Thanks 11
1. Beginning the journey **12**
1.1. Getting the hardware ready 12
1.2. Installing the RISC-V toolchain 16
1.3. Installing OpenOCD 18
1.4. Let’s toggle a GPIO output (RISC-V) 19
1.5. Analyzing the source code 24
1.6. Assemble, link, and program 43
1.7. Final thoughts on chapter 1 51
2. Kill the bugs, align it and tidy up **53**
2.1. Kill the bugs with GDB 53
2.2. The elephant in the room (alignment) 57
2.3. Tidying up by using the preprocessor 62
2.4. Splitting the code into different files 63
2.5. Subroutine library to initialize GPIO pins "srs_gpios.S" 70

2.6. Subsystem Resets lib. "srs_resets.S" ... 80
2.7. The main program "02_toggle_tidy.S" ... 84
2.8. The "ass_link_program_min.sh" script ... 86
2.9. What to do if everything goes wrong ... 88
2.10. Final thoughts on chapter 2 ... 88
3. The clock is ticking ... **90**
3.1. Overview ... 90
3.2. Watching the clock ... 93
3.3. Switching to the crystal oscillator ... 99
3.4. Multiplying the frequency with a PLL ... 102
3.5. Peripheral clock generator ... 110
3.6. ADC clock generator ... 112
3.7. Init clocks ... 113
3.8. LED in hyperstress (hyperspace?) ... 113
3.9. 00_template.S ... 115
3.10. Final thoughts on chapter 3 ... 116
4. Serial ... **117**
4.1. Overview ... 117
4.2. Initializing the UART ... 120
4.3. Read and writing bytes from/to serial ... 129
4.4. Echo, Echo, Echo ... 133
4.5. Serial.print() ... 139
4.6. Peeking at register and memory ... 143
4.7. What about UART1? ... 152
4.8. Final thoughts on chapter 4 ... 152
5. "Make It": Pico2Terminal ... **155**

5.1. Using an HDMI monitor ... 156
5.2. Memory output on the HDMI monitor ... 163
5.3. Using a Keyboard ... 170
5.4. Terminal ... 176
5.5. Final thoughts on chapter 5 ... 183
6. Digital and analog input ... 185
6.1. GPIOx as digital input ... 185
6.2. Analog input using an ADC ... 194
6.3. Final thoughts on chapter 6 ... 211
7. Timers and analog output (PWM) ... 212
7.1. Timer/Counter overview ... 212
7.2. The two system timers TIMER0 and TIMER1 ... 213
7.3. RISC-V platform timer ... 223
7.4. PWM ... 227
7.5. Final thoughts on chapter 7 ... 250
8. Interrupts ... 252
8.1. Overview 252
8.2. RP2350's Hazard3 Interrupts ... 257
8.3. An alarming Rpt interrupt ... 269
8.4. External interrupts: GPIO interrupt ... 276
8.5. External UART interrupt ... 286
8.6. Playing a MIDI song with DDS and an RPT interrupt ... 289
8.7. Direct vs Vectored Interrupt Handling ... 301
8.8. Final thoughts on chapter 8 ... 302
9. "Make It": Pico2Pong ... 303
9.1. Building the hardware ... 303

9.2. Arduino Software for the HDMI Pico2 306
9.3. The game 311
9.4. Final thoughts on chapter 9 329
10. Memory and DMA 331
10.1. The memory map 331
10.2. Stack 335
10.3. The linker script and boundaries 339
10.4. Copying ".data" and ".bss" section to SRAM 343
10.5. DMA: Copy a memory block m2m 348
10.6. DMA: Memory to PWM 359
10.7. DMA: Working with two DMA channels 363
10.8. Final thoughts on chapter 10 369
11. PIO State machines 371
11.1. Overview 371
11.2. Configuring the PIO block and the state machine 376
11.3. The PIO library "srs_pio.S" 383
11.4. New subroutines to handle GPIOs 393
11.5. PIO toggle 396
11.6. WS2812b (Neopixel) interface 404
11.7. 8 Bit parallel output with DMA 412
11.8. I2S hardware interface using PIO 423
11.9. I2S DDS 435
11.10. Final thoughts on chapter 11 452
12. Two hearts 455
12.1. Overview 455
12.2. Wakey wakey 457

12.3. Main program "12_two_cores" 463
12.4. Race conditions and spinlocks 465
12.5. Ring my bell (doorbell interrupts) 466
12.6. Using double buffering to pass data 471
12.7. Global Exclusive Monitor 478
12.8. Final thoughts on chapter 12 483
13. "Make It": Pico2MCWO (Pico2Synth) 485
13.1. Building the hardware 486
13.2. The Interrupt Service Routines 489
13.3. New subroutines and a rainbow 495
13.4. The main program "13_pico2mcwo.S" 508
13.5. Final thoughts on chapter 13 521
Appendix A: ASCII code 523
Appendix B: Pico2 Memory Map 525
Appendix C: RISC-V Assembler 529
C.1. Instruction cheat sheet 529
C.2. Instruction terminology 536
C.3. RISC-V extensions implemented by Hazard3 537
C.4. Remarks 537
C.5. Instructions used per chapter 539
Appendix D: Links 540
Appendix E: Are we hackers? 543
E.1. Analyzing code from the SDK 543
E.2. Rearranging the code 547
E.3. Countercheck with **GDB** 548